

```
In [393... #import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [394... # import database/data
```

```
In [395... conn=sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    where type='table';

"""

tables=pd.read_sql(sql_query,conn)

#create dataframe for each table
for table_name in tables['name']:
    df=pd.read_sql(f"select * FROM {table_name}",conn)
    globals()[f"df_{table_name}"]=df
    print(f"Created dataframe :df_{table_name}")

conn.close
```

Created dataframe :df_db_customer
 Created dataframe :df_db_subscription
 Created dataframe :df_db_support

```
Out[395... <function Connection.close()>
```

```
In [396... conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

Table Name: db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

In [397... **2#DATA CLEANING**

Out[397... 2

In [398... df_db_customer.tail()

Out[398... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody> </tbody>

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [399... df_db_customer.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

In [400...

```
#a.rename col-name
# b.drop columns-interest and pincode
#change datatype -customerid name country state gender
#data standardization -gender
#d.fix missing value-country
```

In [401...

```
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
df_db_customer
```

Out[401...

```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>interests</th>
<th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th>
<th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
</tbody> <tbody> </tbody>

```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None

0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [402...

```
# b. drop columns - interest and pincode

#df_db_customer.drop(df_db_customer.columns[-2:],axis=1)

df_db_customer.drop(columns=['interests','pincode'],inplace=True)
```

In [403...

```
df_db_customer
```

Out[403...

```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th>
<th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th>
<th>19</th> <th>20</th> </tbody> <tbody></tbody>

```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [404...

```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    str
dtypes: str(6)
memory usage: 1.1 KB
```

```
In [405... df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

```
In [406... #change datatype -customerid name country state gender

df_db_customer[['customerid','customer_name','country','state','gender']] = \
df_db_customer[['customerid','customer_name','country','state','gender']].astype
```

```
In [407... #data standardization -gender

df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo
```

```
In [408... df_db_customer['gender'].unique()
```

```
Out[408... array(['Male', 'Female'], dtype=object)
```

```
In [409... #d.fix missing value-country

df_db_customer[df_db_customer['country'].isna()]
```

```
Out[409... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th>
<th>8</th> <th>12</th> </tbody> <tbody></tbody>
```

```
0013-MHZWF durga NaN Delhi Female 1988-12-10
```

```
0015-UOCOJ maya NaN Kathmandu Female 1985-07-07
```

```
0018-NYROU chitra NaN Telangana Female 2004-12-01
```

```
In [410... state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('sta
df_db_customer['country']=df_db_customer['country'].fillna(df_db_customer['state
```

```
In [411... df_db_customer[df_db_customer['country'].isna()]
```

Out[411... `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>`

In [412... `df_db_customer[['country','state']]`

Out[412... `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>`

India	Maharashtra
India	Karnataka
India	Delhi
India	Nagaland
India	Delhi
India	Delhi
India	Meghalaya
India	Rajasthan
Nepal	Kathmandu
Nepal	Kathmandu
India	Maharashtra
India	Karnataka
India	Telangana
India	Meghalaya
India	Uttar Pradesh
India	Delhi
India	Meghalaya
India	Rajasthan
India	Telangana
India	Uttar Pradesh
India	Maharashtra

In [413... `df_db_subscription.head()`


```
Out[413... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

```
In [414... df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null    str
1   subscription_start_date               21 non-null    str
2   subscription_type                     21 non-null    str
3   renewal_date                         21 non-null    str
4   plan_type                            21 non-null    str
5   contract_type                        21 non-null    str
6   cancellation_date                    6 non-null     str
7   cancellation_reason                   6 non-null     str
8   monthly_charges                      21 non-null    float64
9   cltv                                 21 non-null    int64
10  churn_score                          21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

```
In [415... date_col=['subscription_start_date','renewal_date','cancellation_date']

df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date               21 non-null     datetime64[us]
2   subscription_type                     21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                  6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                21 non-null     int64
10  churn_score                          21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB

```

```

In [416... df_db_subscription[['customerid','subscription_type','plan_type','contract_type']]
df_db_subscription[['customerid','subscription_type','plan_type','contract_type']]

```

```

In [417... df_db_subscription.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     object
1   subscription_start_date               21 non-null     datetime64[us]
2   subscription_type                     21 non-null     object
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     object
5   contract_type                        21 non-null     object
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                  6 non-null      object
8   monthly_charges                      21 non-null     float64
9   cltv                                21 non-null     int64
10  churn_score                          21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB

```

```

In [ ]:

```

```

In [418... df_db_support.head()

```

```
Out[418... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

```
In [419... df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

```
In [420... df_db_support[['customerid', 'complaint_date', 'escalations', 'comment']] = \
df_db_support[['customerid', 'complaint_date', 'escalations', 'comment']].astype(o
```

```
In [421... df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

```
In [422... df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [423... df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
dtypes: int64(1), object(3)
memory usage: 420.0+ bytes
```

```
In [424... df_db_support['complaint_date']=pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     object
1   complaint_date  9 non-null     datetime64[us]
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
dtypes: datetime64[us](1), int64(1), object(2)
memory usage: 420.0+ bytes
```

```
In [ ]:
```

```
In [425... #3.Feature Engineering and data Analysis
```

```
In [426... df_db_subscription['churn_flag']=np.where(df_db_subscription['cancellation_date'
```

```
In [427... df_db_subscription.head()
```

```
Out[427... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th>
</tbody> <tbody></tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	C
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	C
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	C
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1



```
In [428... #first fix support table duplicates then merge
```

```
df = (df_db_subscription
      .merge(df_db_customer,on='customerid',how='left')
      .merge(df_db_support,on='customerid',how='left'))
```

```
In [ ]:
```

```
In [429... df.shape
```

```
Out[429... (23, 20)
```

```
In [430... df_db_subscription.shape
```

```
Out[430... (21, 12)
```

```
In [431... df_db_subscription['customerid'].nunique()
```

```
Out[431... 21
```

```
In [432... df_db_customer['customerid'].nunique()
```

```
Out[432... 21
```

```
In [433... df_db_support['customerid'].nunique()
```

```
Out[433... 7
```

```
In [434... df_db_support['customerid'].size
```

Out[434...] 9

In [435...] df_db_support

Out[435...] <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>

0003-MKNFE 2024-08-28 N 60

0003-MKNFE 2024-08-28 Y 10

0013-EXCHZ 2024-01-20 Y 20

0013-MHZWF 2025-03-18 N 90

0013-SMEOE 2024-11-01 N 30

0017-IUDMW 2024-04-10 Y 25

0019-EFAEP 2024-09-27 Y 30

0022-TCJCI 2024-09-13 Y 10

0022-TCJCI 2024-09-14 N 90

In [436...] df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer

In [437...] df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus

In [438...] df_db_support['customerid'].size

Out[438...] 7

In [439...] df = (df_db_subscription
 .merge(df_db_customer,on='customerid',how='left')
 .merge(df_db_support,on='customerid',how='left'))

In [440...] df.shape

Out[440...] (21, 21)

In [441...] df.to_csv('exported_churn_data.csv',index=False)

In []:

In [442...] #data analysi

In [443...] #1.churn rate

In [444...] df.columns

```
Out[444...] Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

```
In [445...] churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ",round(churn_rate,2),"%")
```

Churn Rate = 28.57 %

```
In [446...] # 2.retension Rate
retension_rate = 100- churn_rate
print("Retension Rate = ",round(retension_rate,2),"%")
```

Retention Rate = 71.43 %

```
In [447...] #3.churn by plan type
churn_by_plan=(df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).re

print("churn_by_plan",churn_by_plan)
```

churn_by_plan	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [448...] df.columns
```

```
Out[448...] Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

```
In [449...] #5.Averagw Revenue per user

arpu = df['monthly_charges'].mean()

print('ARPU =',round(arpu,2))
```

ARPU = 18.85

```
In [450...] #6 , Avg customer tennur

#count of days users has used our services:cancellation date else current date

today= pd.Timestamp.today()

df['tenure_days']=np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date'] -df ['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)

df.head()
```

```
avg_tenure= df['tenure_days'].mean()
print("Avg Tenure (Days)=",round(avg_tenure),0)
```

Avg Tenure (Days)= 1493 0

```
In [451... 7#revenue at risk-revenue lost from churned users

revenue_at_risk=df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =",revenue_at_risk)
```

Revenue at Risk (Rs 'K') = 73.94

```
In [452... #8. Escalation Rate

escalation_rate= (df['escalations']=='Y').mean()*100
print("Escalation Rate = ",round(escalation_rate,2),"%")
```

Escalation Rate = 19.05 %

```
In [453... #9.Avg Complaint per user

avg_complaints=df['complaint_count'].sum()/df['customerid'].nunique()
print("Avg Complaints Per User=",round(avg_complaints,2))
```

Avg Complaints Per User= 0.43

```
In [454... #10 correlation escalation vs churn

df['escalations']=np.where(df['escalations'] == 'Y',1,0)#encoding string to int

corr_df =df [['escalations','churn_flag']].dropna()

correlation=corr_df['escalations'].corr(df['churn_flag'])

print("Correlation between escalation Vs churn is=",round(correlation,2))
```

Correlation between escalation Vs churn is= 0.77

```
In [455... #11.Create a column using existing column -churn risk

conditions = [

    (df['churn_score']<50),
    (df['churn_score']>=50) & (df['churn_score']<70),
    (df['churn_score']>=70)
]

choices =['low','med','high']

df['churn_risk']=np.select(conditions,choices,default='unknow')
```

```
In [456... df[['churn_risk','churn_score']].tail()
```



```
Out[456... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>churn_risk</th> <th>churn_score</th> </thead> <tbody>
<th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody>
<tbody></tbody>
med 62
low 27
high 99
low 7
low 47
```

```
In [457... # visualization using matplotlib
```

```
In [458... df_visual=df.copy()
```

```
In [459... df_visual.columns
```

```
Out[459... Index(['customerid', 'subscription_start_date', 'subscription_type',
        'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
        'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
        'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
        'complaint_date', 'escalations', 'csat_score', 'complaint_count',
        'tenure_days', 'churn_risk'],
        dtype='str')
```

```
In [460... #4.1 Monthly Churn Trend(Time Series KPI)

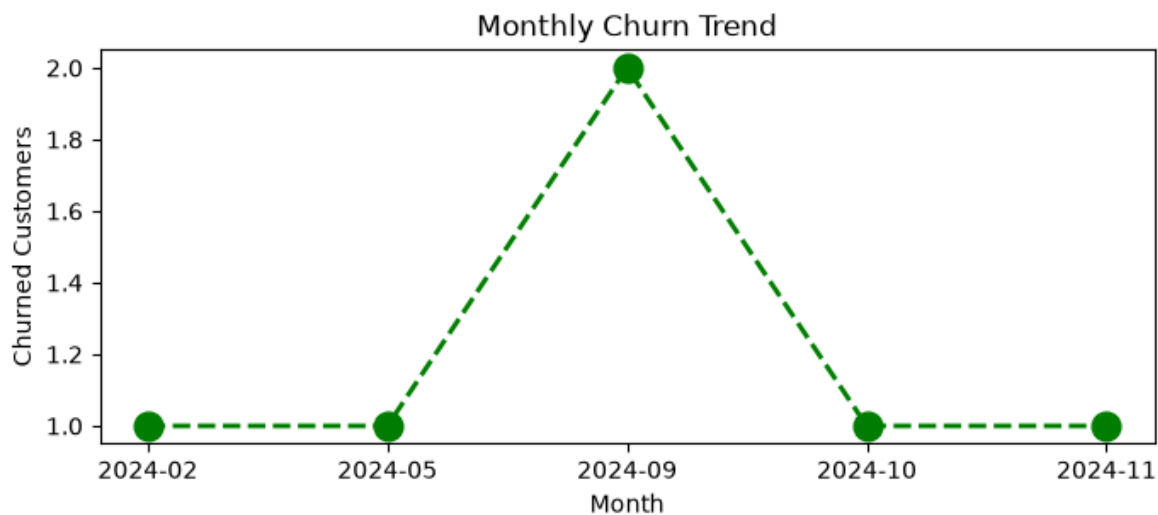
df_visual['cancellation_month']=df_visual['cancellation_date'].dt.to_period('M')

churn_trend =df_visual[df_visual['churn_flag']==1].groupby('cancellation_month')

plt.figure(figsize=(8,3))

plt.plot(churn_trend.index.astype('str'),churn_trend.values,color='green', marke

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



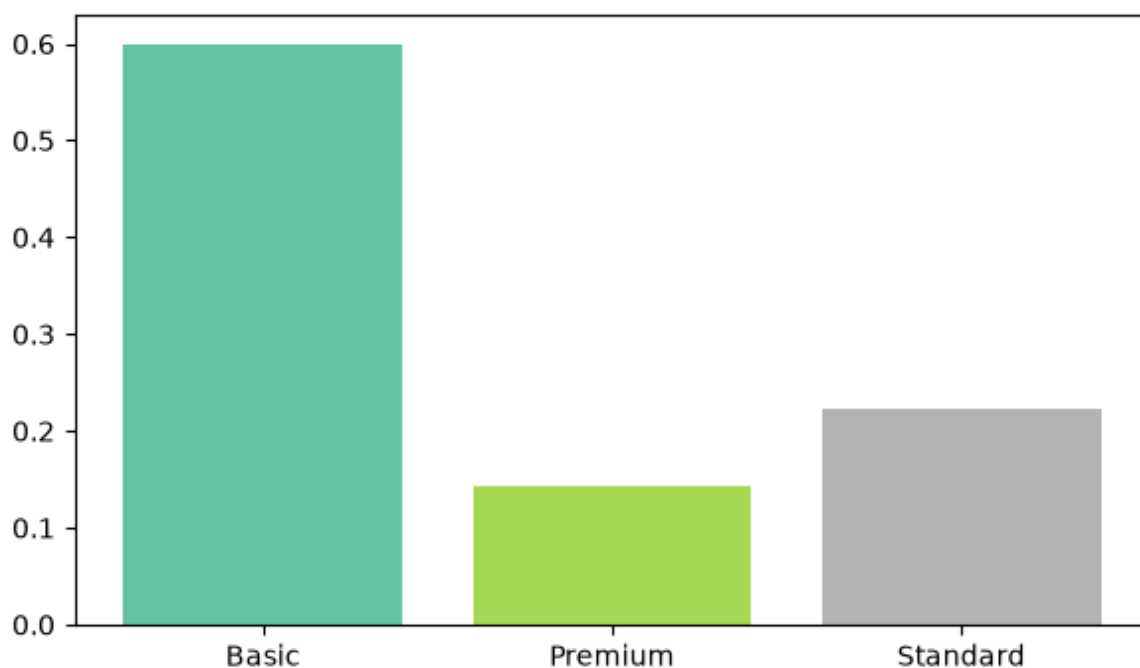
In [461...

```
#4.2 Churn by plan type
churn_plan=df_visual.groupby('plan_type')['churn_flag'].mean()

colors=plt.cm.Set2(np.linspace(0,1,len(churn_plan)))

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index,churn_plan.values,color=colors)
plt.show()
```



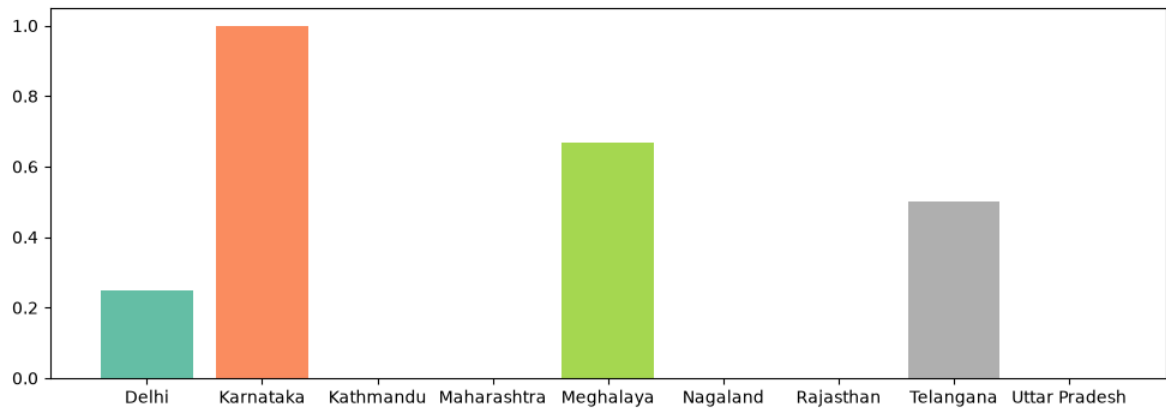
In [462...

```
# 4.3 churn by state
churn_plan=df_visual.groupby('state')['churn_flag'].mean()

colors=plt.cm.Set2(np.linspace(0,1,len(churn_plan)))

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index,churn_plan.values,color=colors)
plt.show()
```



In [463... *# Visualization using seaborn*

In [464... *#encoding-convert str to numeric so that we can find corr between feature*
 df_visual.columns

Out[464... Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk', 'cancellation_month'], dtype='str')

In [465... df_encoded.head()

Out[465... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
 <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
 <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody>
 <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
 </tbody>

2	0	12	0	1	0
1	0	91	1	0	1
0	1	34	0	1	0
1	0	8	0	1	0
2	1	88	1	0	1

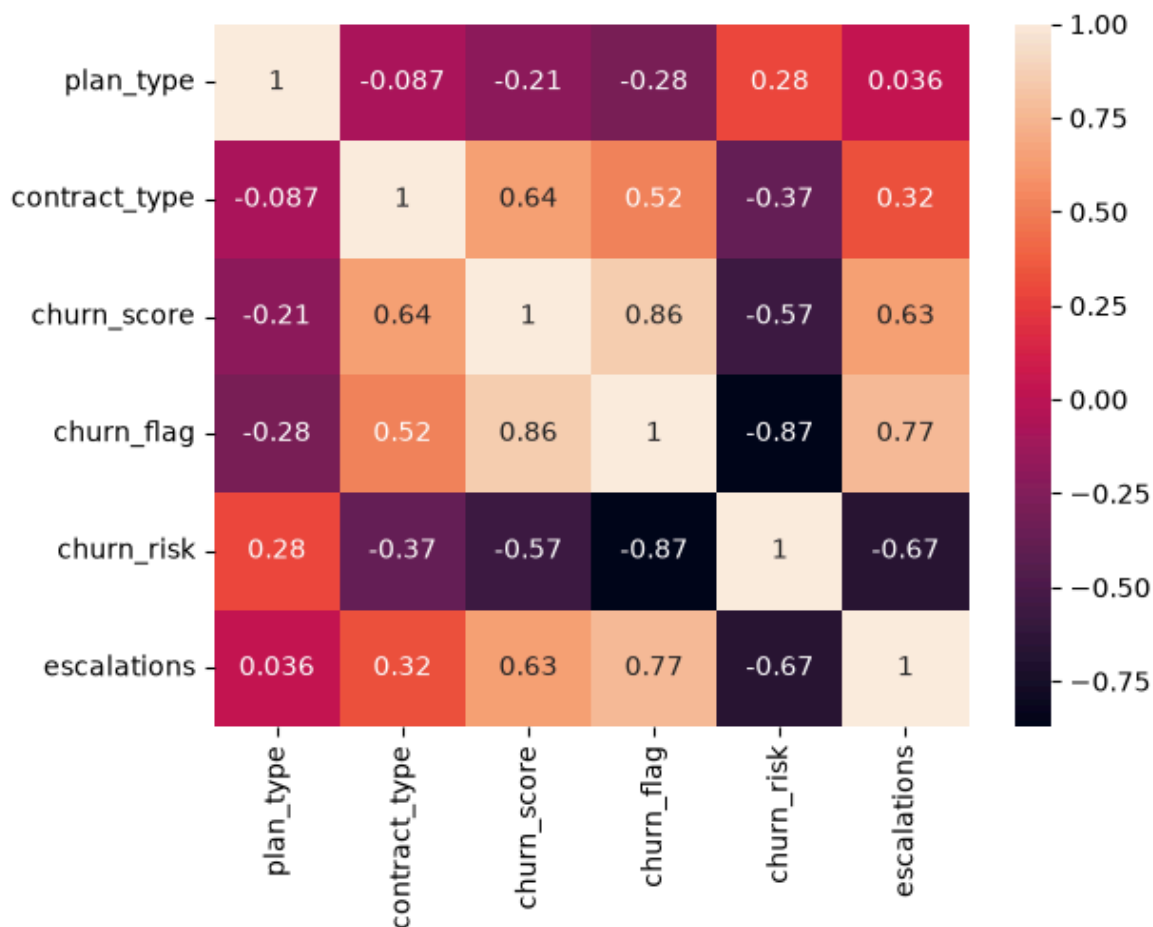
In [466... *#incorrect method of encoding -as numbers are not assigned based on priority*
 df_encoded=df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'cancellation_month']]
 categorical_cols=['plan_type', 'contract_type', 'churn_risk']
 for col in categorical_cols:
 df_encoded[col] = df_encoded[col].astype('category').cat.codes

In []:

```
In [467... #Heatmap(correlation matrix)

sns.heatmap(df_encoded.corr(),annot=True)
```

Out[467... <Axes: >



```
In [469... #correct method of encoding -as numbers are not assigned based on priority
df_encoded=df_visual[['plan_type','contract_type','churn_score','churn_flag','churn_risk','escalations']]

order_mappings={
    'plan_type':['Basic','Standard','Premium'],
    'contract_type':['Monthly','Annual'],
    'churn_risk':['low','med','high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'),categories=order)
```

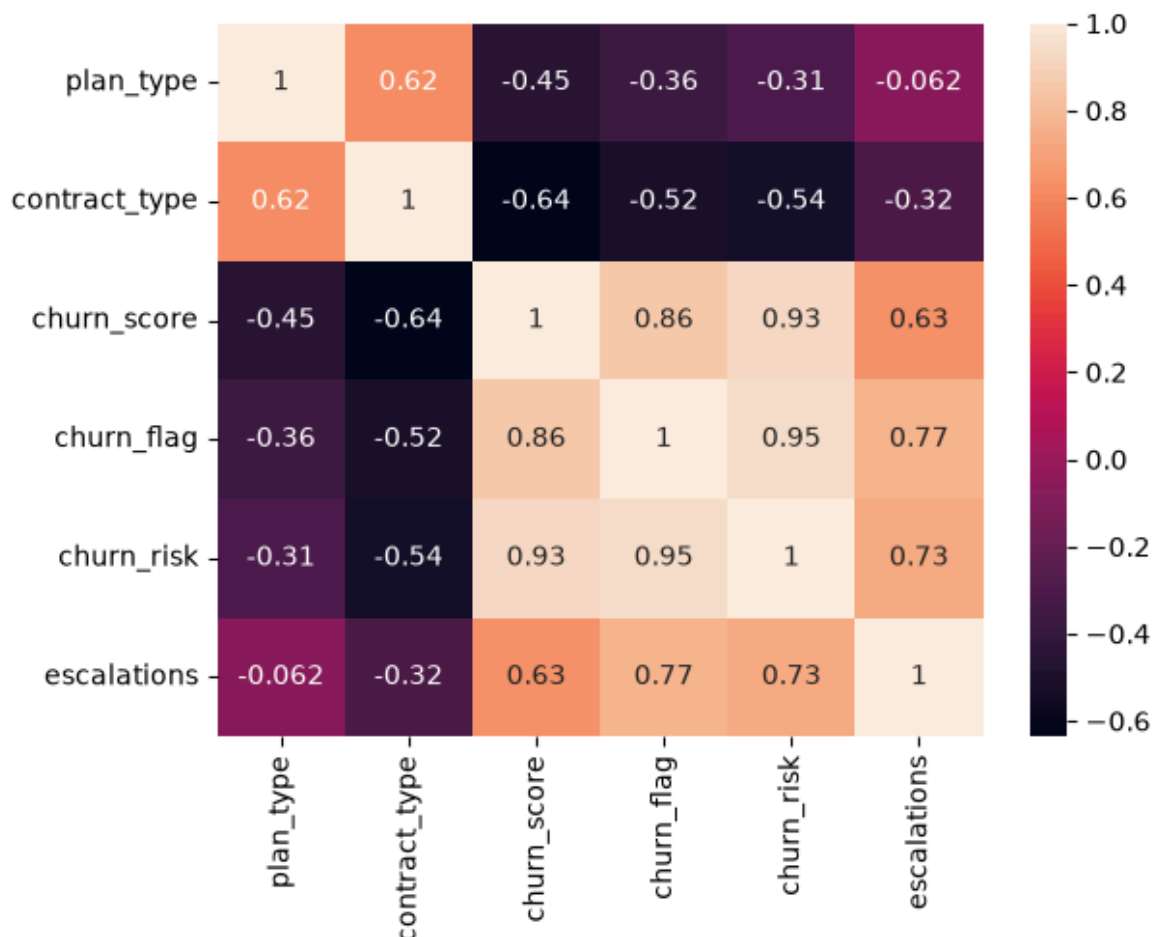
```
In [470... df_visual[['plan_type','contract_type','churn_score','churn_flag','churn_risk','escalations']]
```

Out[470... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

Standard	Annual	12	0	low	0
Premium	Annual	91	1	high	1
Basic	Monthly	34	0	low	0
Premium	Annual	8	0	low	0
Standard	Monthly	88	1	high	1

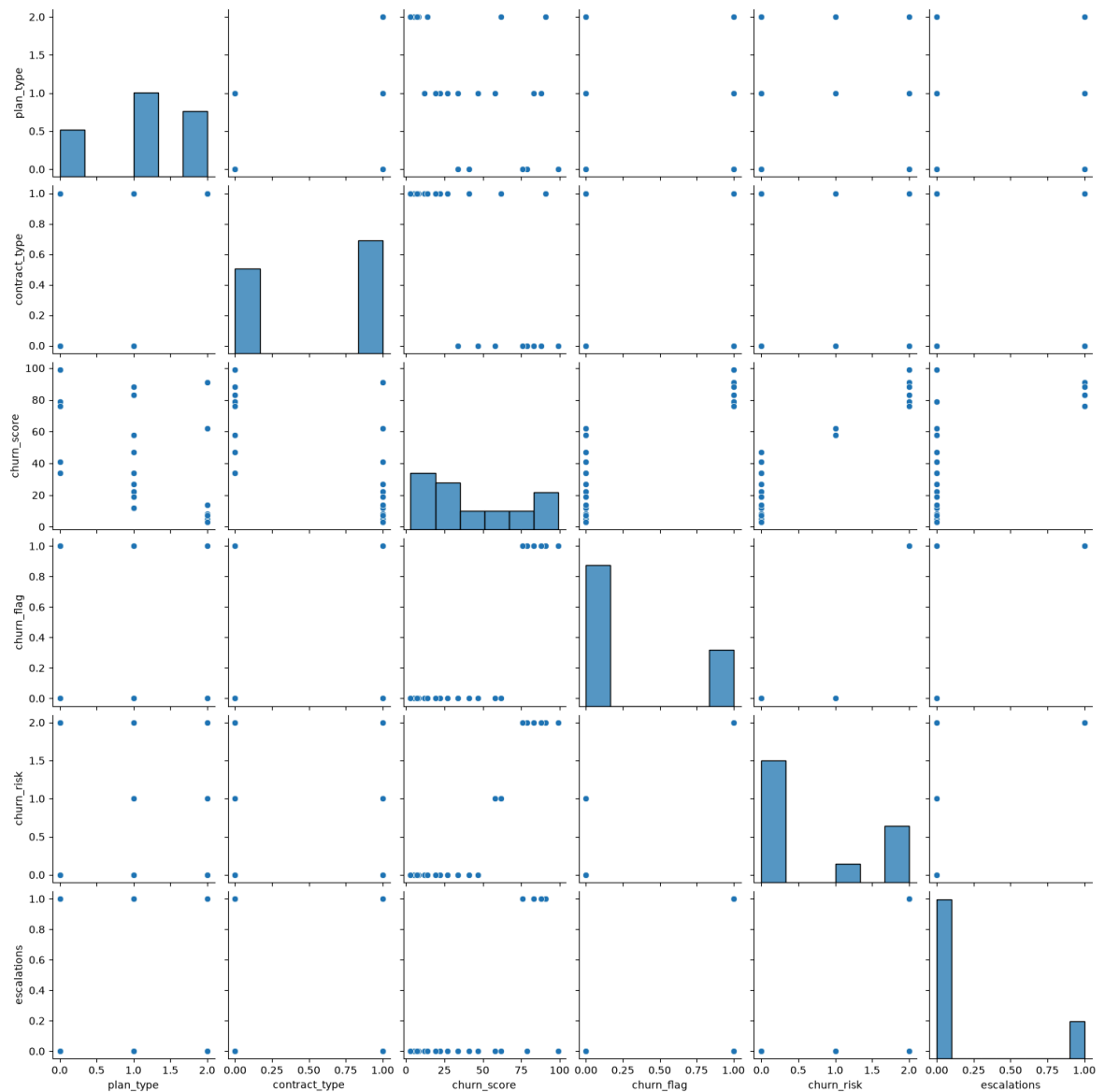
In [471... *#Heatmap(correlation matrix)*
sns.heatmap(df_encoded.corr(),annot=True)

Out[471... <Axes: >



In [473... *#pairplot -relationship in a dataset*
sns.pairplot(df_encoded)

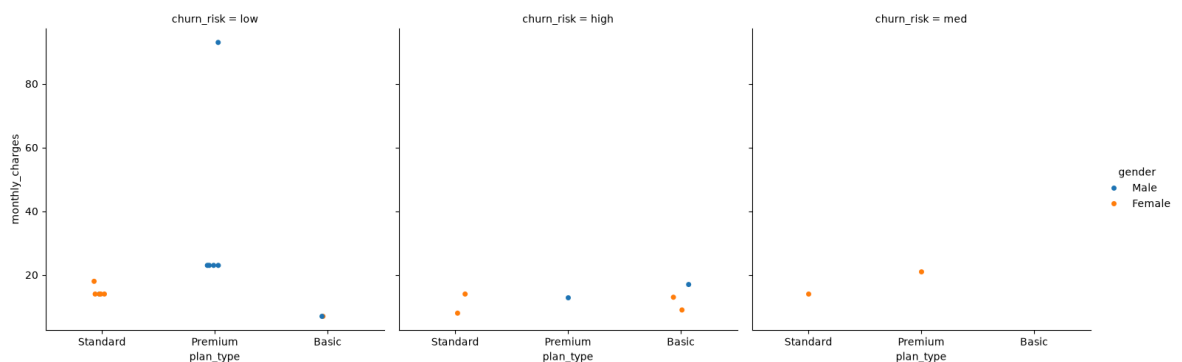
Out[473... <seaborn.axisgrid.PairGrid at 0x1ca051c5010>



In [474... *#catplt/Facegrid plot-multi-dim comparison*

```
sns.catplot(data=df_visual,
             x='plan_type',
             y='monthly_charges',
             hue='gender',
             col='churn_risk')
```

Out[474... <seaborn.axisgrid.FacetGrid at 0x1ca06a53a10>



In []:

In []: *#pivot tables*

```
In [480... pd.pivot_table(
    df_visual,
    values='churn_flag',
    index='plan_type',
    aggfunc='mean'
).reset_index()
```

Out[480... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Basic	0.600000
-------	----------

Premium	0.142857
---------	----------

Standard	0.222222
----------	----------

In [481... df_visual.columns

Out[481... Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk', 'cancellation_month'], dtype='str')

In []:

```
In [484... pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc={
        'monthly_charges': 'sum',
        'customerid': 'nunique',
        'churn_flag': 'mean'
    }
)
```

Out[484... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> <th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000	5	52.95
0.142857	7	218.93
0.222222	9	123.91

In []: *#working with SQL in python(pandas)*

In [493...

```
#create db in sql

conn = sqlite3.connect('test_databases.sqlite')

#table details

conn.execute("CREATE TABLE users(first_name TEXT,country TEXT,budget INTEGER)")

#commit and save
conn.commit()
```

In [494...

```
#insert data

cursor = conn.cursor()

#write sql query to insert records in sql table
cursor.execute(
    """
        INSERT INTO users VALUES
            ('Madhav','india',5000),
            ('Rishabh','Germany',2500),
            ('Vishakha','india',3500)
    """
)

#commit and save

conn.commit()

print("Data Inserted sucessfully")
```

Data Inserted sucessfully

In [495...

```
#check inserted data in table

conn = sqlite3.connect('test_databases.sqlite')
query="""SELECT * FROM users """

df_results = pd.read_sql(query,conn)

df_results.head()
```

Out[495...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>first_name</th> <th>country</th> <th>budget</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>
```

Madhav	india	5000
--------	-------	------

Rishabh	Germany	2500
---------	---------	------

Vishakha	india	3500
----------	-------	------

In [496...

```
#aggregation

query="""
    SELECT country,sum(budget) as total_budget
    From users
```


Group BY country

"""

```
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[496... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody> </tbody>

Germany 2500

india 8500

In [497... task is over close conn
conn.close()

In []:

In []: